

```

        if (!encontrado) Console.WriteLine("Libro no encontrado en el sistema.");
        Console.ReadKey();
    }

    static void ListarLibros()
    {
        Console.Clear();
        Console.WriteLine("== LISTA DE LIBROS REGISTRADOS ==");
        if (contadorLibros == 0) Console.WriteLine("No hay libros en memoria.");

        for (int i = 0; i < contadorLibros; i++)
        {
            Console.WriteLine($"{i + 1}. [{libros[i].codigo}] {libros[i].titulo} - {libros[i].autor}
(Dispo: {libros[i].cantidadEjemplares})");
        }
        Console.ReadKey();
    }

    static void EliminarLibro()
    {
        Console.Clear();
        Console.WriteLine("== ELIMINAR LIBRO ==");
        Console.Write("Ingrese el código del libro a borrar: ");
        string cod = Console.ReadLine().Trim();

        int indiceEncontrado = -1;
        for (int i = 0; i < contadorLibros; i++)
        {
            if (libros[i].codigo.ToLower() == cod.ToLower())
            {
                indiceEncontrado = i;
                break;
            }
        }

        if (indiceEncontrado != -1)
        {
            // Mover los elementos hacia atrás para cubrir el hueco (Lógica de vectores
            tradicional)
            for (int i = indiceEncontrado; i < contadorLibros - 1; i++)
            {
                libros[i] = libros[i + 1];
            }
            contadorLibros--;
            Console.WriteLine("Libro removido del registro.");
        }
        else Console.WriteLine("El libro especificado no existe.");
    }

```

```

    Console.ReadKey();
}

// =====
// LÓGICA MÓDULO B: USUARIOS (Sección 3.1)
// =====
static void RegistrarUsuario()
{
    Console.Clear();
    Console.WriteLine("== REGISTRAR USUARIO ==");

    if (contadorUsuarios >= 5)
    {
        Console.WriteLine("Error: Límite máximo de 5 usuarios alcanzado.");
        Console.ReadKey();
        return;
    }

    Usuario nuevoUsuario = new Usuario();

    // Validación carné de 8 dígitos numéricos
    while (true)
    {
        Console.Write("Carné (8 dígitos numéricos): ");
        string car = Console.ReadLine().Trim();

        bool esNumerico = true;
        foreach (char c in car)
        {
            if (!char.IsDigit(c)) esNumerico = false;
        }

        if (car.Length == 8 && esNumerico)
        {
            nuevoUsuario.carne = car;
            break;
        }
        Console.WriteLine("Formato erróneo. Deben ser exactamente 8 números.");
    }

    Console.Write("Nombre Completo: ");
    nuevoUsuario.nombreCompleto = Console.ReadLine().Trim();
    Console.Write("Carrera: ");
    nuevoUsuario.carrera = Console.ReadLine().Trim();

    // Validación de correo con '@' y '.' posterior
    while (true)
    {

```

```

        Console.WriteLine("Correo Electrónico: ");
        string mail = Console.ReadLine().Trim();
        int arrobaldx = mail.IndexOf('@');
        int puntoldx = mail.LastIndexOf('.');

        if (arrobaldx != -1 && puntoldx != -1 && puntoldx > arrobaldx)
        {
            nuevoUsuario.correoElectronico = mail;
            break;
        }
        Console.WriteLine("Correo inválido. Debe contener '@' y un '.' después del mismo.");
    }

```

```

        Console.WriteLine("Teléfono: ");
        nuevoUsuario.telefono = Console.ReadLine().Trim();
        nuevoUsuario.estado = "Activo"; // Por defecto al registrarse

```

```

        usuarios[contadorUsuarios] = nuevoUsuario;
        contadorUsuarios++;
        Console.WriteLine("Usuario registrado con éxito.");
        Console.ReadKey();
    }

```

```

static void BuscarUsuario()
{
    Console.Clear();
    Console.WriteLine("== BUSCAR USUARIO ==");
    Console.WriteLine("Ingrese Carné o Nombre a buscar: ");
    string filtro = Console.ReadLine().Trim().ToLower();

    bool hallado = false;
    for (int i = 0; i < contadorUsuarios; i++)
    {
        if (usuarios[i].carne.ToLower() == filtro ||
            usuarios[i].nombreCompleto.ToLower().Contains(filtro))
        {
            Console.WriteLine($"Carné: {usuarios[i].carne}");
            Console.WriteLine($"Nombre: {usuarios[i].nombreCompleto}");
            Console.WriteLine($"Carrera: {usuarios[i].carrera}");
            Console.WriteLine($"Correo: {usuarios[i].correoElectronico}");
            Console.WriteLine($"Estado: {usuarios[i].estado}");
            hallado = true;
        }
    }

    if (!hallado) Console.WriteLine("No se encontraron coincidencias.");
    Console.ReadKey();
}

```

```

}

static void ListarUsuarios()
{
    Console.Clear();
    Console.WriteLine("== LISTA TOTAL DE USUARIOS ==");
    if (contadorUsuarios == 0) Console.WriteLine("No hay usuarios registrados.");

    for (int i = 0; i < contadorUsuarios; i++)
    {
        Console.WriteLine($"- {usuarios[i].carne} | {usuarios[i].nombreCompleto} | {usuarios[i].carrera} ({usuarios[i].estado})");
    }
    Console.ReadKey();
}

// =====
// LÓGICA MÓDULO C: PRÉSTAMOS (Sección 3.1)
// =====
static void RegistrarPrestamo()
{
    Console.Clear();
    Console.WriteLine("== REGISTRAR PRÉSTAMO ==");

    if (contadorPrestamos >= 10)
    {
        Console.WriteLine("Límite de 10 préstamos simultáneos lleno.");
        Console.ReadKey();
        return;
    }

    Console.Write("Ingrese Carné del usuario: ");
    string carne = Console.ReadLine().Trim();
    int userIdx = -1;
    for (int i = 0; i < contadorUsuarios; i++)
    {
        if (usuarios[i].carne == carne) userIdx = i;
    }

    if (userIdx == -1 || usuarios[userIdx].estado == "Inactivo")
    {
        Console.WriteLine("Usuario no existe o está Inactivo.");
        Console.ReadKey();
        return;
    }

    Console.Write("Ingrese Código del Libro: ");
    string cod = Console.ReadLine().Trim();

```

```

int libroIdx = -1;
for (int i = 0; i < contadorLibros; i++)
{
    if (libros[i].codigo == cod) libroIdx = i;
}

if (libroIdx == -1)
{
    Console.WriteLine("El libro especificado no existe.");
    Console.ReadKey();
    return;
}

if (libros[libroIdx].cantidadEjemplares <= 0)
{
    Console.WriteLine("No hay existencias disponibles de este libro actualmente.");
    Console.ReadKey();
    return;
}

Prestamo nuevoP = new Prestamo();
nuevoP.carneUsuario = carne;
nuevoP.codigoLibro = cod;

// Validación básica de fechas por longitud y separadores (ej: dd/mm/yyyy)
while (true)
{
    Console.Write("Fecha del Préstamo (dd/mm/yyyy): ");
    string f1 = Console.ReadLine().Trim();
    if (f1.Length == 10 && f1[2] == '/' && f1[5] == '/')
    {
        nuevoP.fechaPrestamo = f1;
        break;
    }
    Console.WriteLine("Formato de fecha inválido.");
}

while (true)
{
    Console.Write("Fecha Estimada de Devolución (dd/mm/yyyy): ");
    string f2 = Console.ReadLine().Trim();
    if (f2.Length == 10 && f2[2] == '/' && f2[5] == '/')
    {
        nuevoP.fechaEstimadaDevolucion = f2;
        break;
    }
    Console.WriteLine("Formato de fecha inválido.");
}

```

```

nuevoP.estado = "Activo";

// Modificar inventario en memoria
libros[libroIdx].cantidadEjemplares--;

prestamos[contadorPrestamos] = nuevoP;
contadorPrestamos++;

Console.WriteLine("¡Préstamo concedido con éxito!");
Console.ReadKey();
}

static void RegistrarDevolucion()
{
    Console.Clear();
    Console.WriteLine("== CONTROL DE DEVOLUCIONES ==");
    Console.Write("Ingrese Carné del usuario: ");
    string carne = Console.ReadLine().Trim();
    Console.Write("Ingrese Código del libro: ");
    string libro = Console.ReadLine().Trim();

    bool hecho = false;
    for (int i = 0; i < contadorPrestamos; i++)
    {
        if (prestamos[i].carneUsuario == carne && prestamos[i].codigoLibro == libro &&
prestamos[i].estado == "Activo")
        {
            prestamos[i].estado = "Devuelto";

            // Devolver al stock
            for (int j = 0; j < contadorLibros; j++)
            {
                if (libros[j].codigo == libro) libros[j].cantidadEjemplares++;
            }

            Console.WriteLine("Devolución asentada y stock actualizado.");
            hecho = true;
            break;
        }
    }

    if (!hecho) Console.WriteLine("No se encontró ningún préstamo activo con esas
credenciales.");
    Console.ReadKey();
}

static void HistorialPrestamosUsuario()

```

```

{
    Console.Clear();
    Console.WriteLine("== PRÉSTAMOS ACTIVOS POR USUARIO ==");
    Console.Write("Ingrese Carné del usuario: ");
    string carne = Console.ReadLine().Trim();

    int cont = 0;
    for (int i = 0; i < contadorPrestamos; i++)
    {
        if (prestamos[i].carneUsuario == carne && prestamos[i].estado == "Activo")
        {
            Console.WriteLine($"- Libro: {prestamos[i].codigoLibro} | Prestado el:
{prestamos[i].fechaPrestamo} | Devuelve el: {prestamos[i].fechaEstimadaDevolucion}");
            cont++;
        }
    }

    if (cont == 0) Console.WriteLine("El usuario no posee préstamos activos.");
    Console.ReadKey();
}

```

```

static void ActualizarEstadoPrestamo()
{
    Console.Clear();
    Console.WriteLine("== MODIFICAR ESTADO DEL PRÉSTAMO ==");
    Console.Write("Ingrese Carné del usuario: ");
    string carne = Console.ReadLine().Trim();
    Console.Write("Ingrese Código del libro: ");
    string libro = Console.ReadLine().Trim();

    for (int i = 0; i < contadorPrestamos; i++)
    {
        if (prestamos[i].carneUsuario == carne && prestamos[i].codigoLibro == libro)
        {
            Console.WriteLine($"Estado actual: {prestamos[i].estado}");
            Console.Write("Ingrese nuevo estado (Activo/Devuelto/Vencido): ");
            prestamos[i].estado = Console.ReadLine().Trim();
            Console.WriteLine("Estado cambiado exitosamente.");
            Console.ReadKey();
            return;
        }
    }

    Console.WriteLine("Préstamo no localizado.");
    Console.ReadKey();
}

```

```

// =====
// MANEJO DE ARCHIVOS: PERSISTENCIA (Sección 4.4)

```

```

// =====
static void GuardarDatosEnArchivos()
{
    try
    {
        // 1. Guardar Libros (Formato CSV)
        using (StreamWriter sw = new StreamWriter(archivoLibros, false))
        {
            for (int i = 0; i < contadorLibros; i++)
            {

sw.WriteLine($"{libros[i].codigo},{libros[i].titulo},{libros[i].autor},{libros[i].editorial},{libros[i].anio
Publicacion},{libros[i].categoria},{libros[i].cantidadEjemplares}");
            }
        }

        // 2. Guardar Usuarios (Formato TXT separado por pipelines '|')
        using (StreamWriter sw = new StreamWriter(archivoUsuarios, false))
        {
            for (int i = 0; i < contadorUsuarios; i++)
            {

sw.WriteLine($"{usuarios[i].carne}|{usuarios[i].nombreCompleto}|{usuarios[i].carrera}|{usuari
os[i].correoElectronico}|{usuarios[i].telefono}|{usuarios[i].estado}");
            }
        }

        // 3. Guardar Préstamos (Formato TXT separado por pipelines '|')
        using (StreamWriter sw = new StreamWriter(archivoPrestamos, false))
        {
            for (int i = 0; i < contadorPrestamos; i++)
            {

sw.WriteLine($"{prestamos[i].carneUsuario}|{prestamos[i].codigoLibro}|{prestamos[i].fechaPr
estamo}|{prestamos[i].fechaEstimadaDevolucion}|{prestamos[i].estado}");
            }
        }

        Console.WriteLine("Datos respaldados correctamente en la carpeta Data/.");
    }
    catch (Exception ex)
    {
        Console.WriteLine("Error al guardar: " + ex.Message);
    }
}

static void CargarDatosDesdeArchivos()
{

```



```

try
{
    // Cargar Libros
    if (File.Exists(archivoLibros))
    {
        string[] lineas = File.ReadAllLines(archivoLibros);
        contadorLibros = 0;
        foreach (string l in lineas)
        {
            if (string.IsNullOrEmpty(l)) continue;
            string[] datos = l.Split(',');
            if (datos.Length == 7 && contadorLibros < 10)
            {
                libros[contadorLibros].codigo = datos[0];
                libros[contadorLibros].titulo = datos[1];
                libros[contadorLibros].autor = datos[2];
                libros[contadorLibros].editorial = datos[3];
                libros[contadorLibros].anioPublicacion = int.Parse(datos[4]);
                libros[contadorLibros].categoria = datos[5];
                libros[contadorLibros].cantidadEjemplares = int.Parse(datos[6]);
                contadorLibros++;
            }
        }
    }

    // Cargar Usuarios
    if (File.Exists(archivoUsuarios))
    {
        string[] lineas = File.ReadAllLines(archivoUsuarios);
        contadorUsuarios = 0;
        foreach (string l in lineas)
        {
            if (string.IsNullOrEmpty(l)) continue;
            string[] datos = l.Split('|');
            if (datos.Length == 6 && contadorUsuarios < 5)
            {
                usuarios[contadorUsuarios].carne = datos[0];
                usuarios[contadorUsuarios].nombreCompleto = datos[1];
                usuarios[contadorUsuarios].carrera = datos[2];
                usuarios[contadorUsuarios].correoElectronico = datos[3];
                usuarios[contadorUsuarios].telefono = datos[4];
                usuarios[contadorUsuarios].estado = datos[5];
                contadorUsuarios++;
            }
        }
    }

    // Cargar Préstamos

```

```

if (File.Exists(archivoPrestamos))
{
    string[] lineas = File.ReadAllLines(archivoPrestamos);
    contadorPrestamos = 0;
    foreach (string l in lineas)
    {
        if (string.IsNullOrEmpty(l)) continue;
        string[] datos = l.Split('|');
        if (datos.Length == 5 && contadorPrestamos < 10)
        {
            prestamos[contadorPrestamos].carneUsuario = datos[0];
            prestamos[contadorPrestamos].codigoLibro = datos[1];
            prestamos[contadorPrestamos].fechaPrestamo = datos[2];
            prestamos[contadorPrestamos].fechaEstimadaDevolucion = datos[3];
            prestamos[contadorPrestamos].estado = datos[4];
            contadorPrestamos++;
        }
    }
}
catch
{
    // Si da error de parseo al leer archivos vacíos o corruptos, inicia en limpio
    silenciosamente
}
}
}
}

```